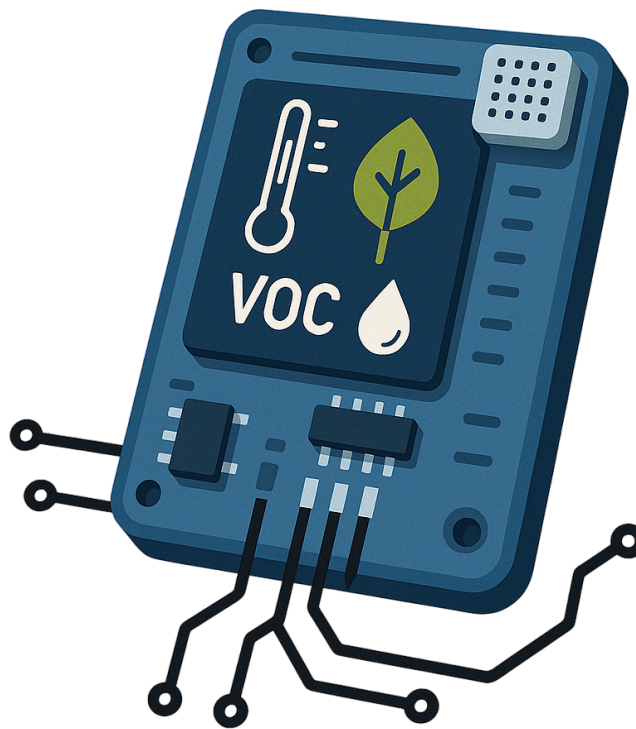


Sistema di Monitoraggio Ambientale

Progettazione di Sistemi Embedded



Alessio Manuelli - Mattia Bernini - Samuele Bartorelli

Indice

1. Introduzione	3
2. Obiettivi del progetto	3
3. Architettura del Sistema Embedded	3
3.2 Sensori	4
3.2.1 Sensore BME688	4
3.3 Interfaccia Utente/Attuatori	4
3.3.1 Display Terasic LT24	4
3.3.2 MicroSD Card Adapter	4
3.3.3 Switch	4
4. Architettura del Computer	5
4.1 Moduli Presenti nel Computer	5
4.2 Utilizzo delle Risorse	6
4.3 Timing Analysis	7
5. Struttura del Software	7
5.1 Misure sensore BME688	7
5.2 Logging su SD	9
5.3 Controllo display LT24	11
5.5 Controllo modalità con switch	12
5.6 Memoria	12
6. Verifica Raggiungimento degli Obiettivi	13

1. Introduzione

Lo scopo del progetto è la realizzazione di un sistema di monitoraggio ambientale. In particolare interessa andare a rilevare valori di temperatura, umidità e pressione dell'ambiente circostante.

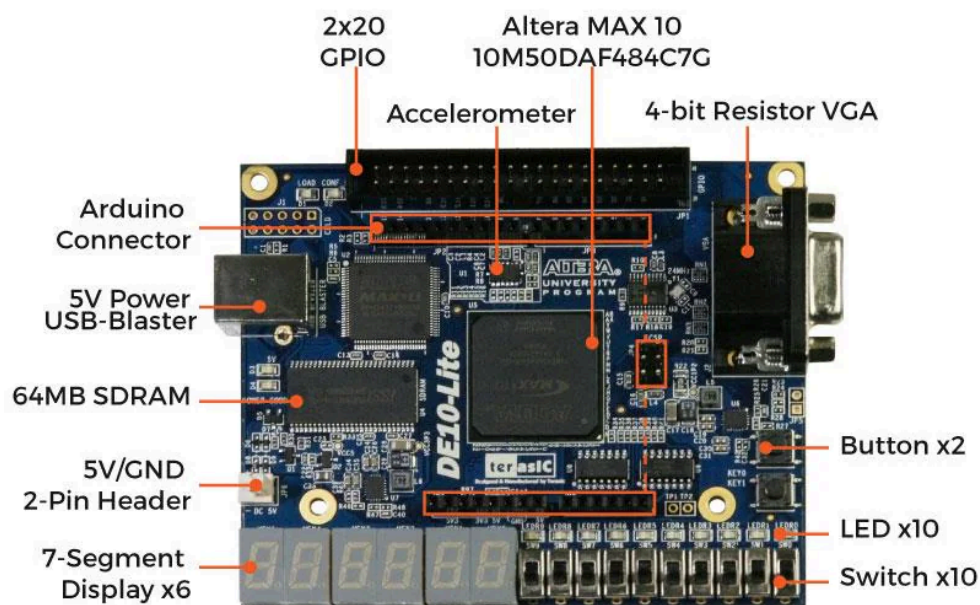
2. Obiettivi del progetto

Di seguito vengono riportati gli obiettivi del progetto:

1. Visualizzazione su display LT24 dei valori di temperatura, umidità e pressione presenti nell'ambiente. In base ad una configurazione degli switch presenti sulla scheda (configurabile dall'utente) vengono visualizzati i valori attuali o i valori massimi/medi/minimi delle varie grandezze.
2. Possibilità di logging su scheda SD dei valori misurati.

3. Architettura del Sistema Embedded

Il progetto utilizza la scheda Terasic DE10-Lite Development Board.



E' basata sulla FPGA MAX10 di Intel. Questa FPGA offre circa 50k Logic Elements, una memoria embedded di 200 kB (blocchi M9k), 4 PLL per la generazione di segnali di clock e molteplici altre risorse.

Di seguito viene riportata la componentistica utilizzata nel progetto.

3.2 Sensori

3.2.1 Sensore BME688

Il sensore BME688 combina rilevamento di temperatura, umidità, pressione atmosferica e qualità dell'aria, grazie a un sensore di gas MOS incorporato. È particolarmente utilizzato in applicazioni di monitoraggio ambientale e domotica. Per quanto riguarda la comunicazione, il sensore utilizza il protocollo I²C.



3.3 Interfaccia Utente/Attuatori

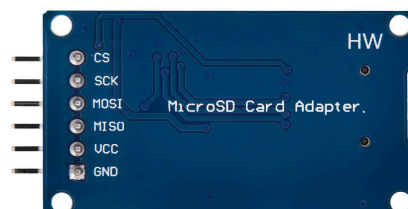
3.3.1 Display Terasic LT24

Il display LT24 della Terasic è un pannello LCD a colori con risoluzione 240x320 pixel. La gestione del display avviene attraverso segnali di controllo dedicati (come RS, WR, RD), che coordinano la scrittura e la lettura dei dati sul pannello. Per ottimizzare le prestazioni e la gestione dei dati da visualizzare, questi vengono scritti prima in una memoria intermedia, come la SDRAM, utilizzata come buffer. Questa soluzione permette di preparare l'intero contenuto grafico in memoria. Successivamente, i dati vengono trasferiti in blocco dal buffer SDRAM al display LT24.



3.3.2 MicroSD Card Adapter

Il microSD card adapter è un modulo che consente di interfacciare una scheda microSD con un microcontrollore o un sistema embedded per l'archiviazione esterna dei dati. Questo adattatore utilizza il protocollo SPI per la comunicazione seriale con il dispositivo principale.



Nell'ambito del progetto, inizialmente è stata utilizzata una scheda microSD da 4GB formattata in FAT32. Tuttavia, questa configurazione non ha permesso il corretto funzionamento del modulo, probabilmente a causa di limitazioni del firmware o del driver che non supportavano completamente il file system FAT32 o schede di capacità superiore a 2GB. Per ovviare a questo problema, siamo passati ad una microSD da 2GB, formattata in FAT16. Con questa nuova configurazione, il sistema ha iniziato a funzionare correttamente.

3.3.3 Switch



Per gestire il contenuto visualizzato sul display LT24, si utilizzano tre switch configurabili manualmente dall'utente. Questi switch permettono di selezionare tra quattro modalità di visualizzazione diverse, modificando dinamicamente i dati mostrati sullo schermo.

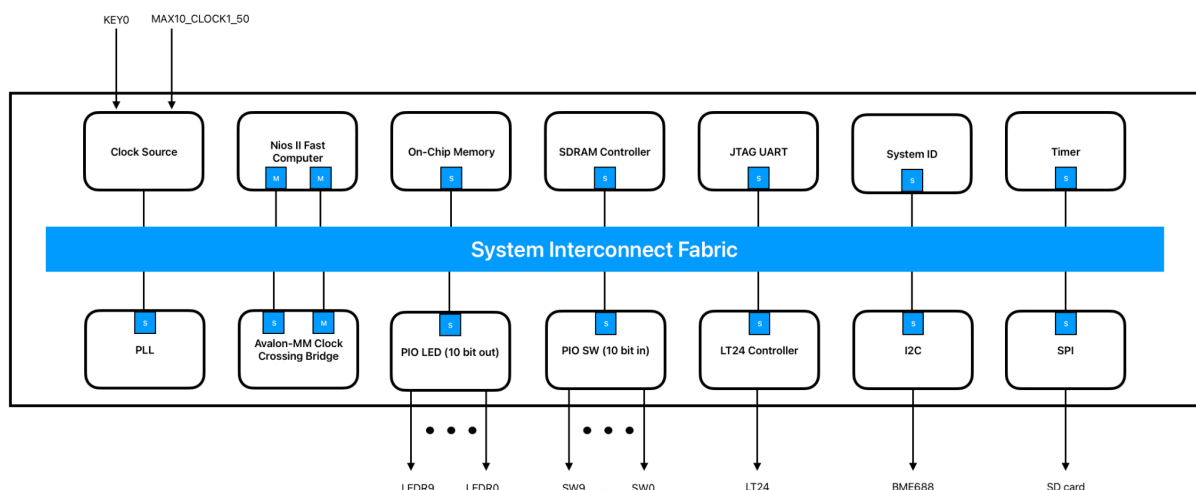
La combinazione delle posizioni degli switch consente di scegliere le seguenti modalità:

1. **Visualizzazione valori attuali:** mostra in tempo reale i valori correnti di temperatura, pressione e umidità.
2. **Visualizzazione valori di temperatura (max, min e medi):** mostra i valori massimi, minimi e medi registrati per la temperatura.
3. **Visualizzazione valori di umidità (max, min e medi):** mostra i valori massimi, minimi e medi dell'umidità relativa.
4. **Visualizzazione valori di pressione (max, min e medi):** mostra i valori massimi, minimi e medi della pressione atmosferica.

Le unità di misura utilizzate nella visualizzazione di temperatura e pressione sono gradi (°C) e atmosfere (atm). Viene visualizzata inoltre l'umidità relativa (%).

4. Architettura del Computer

Il computer è composto da IP presenti nella libreria standard di Platform Designer. Di seguito l'architettura completa del computer utilizzato nel progetto.



4.1 Moduli Presenti nel Computer

Di seguito viene riportata la descrizione di tutti i moduli presenti all'interno del computer.

- **Clock Source:** rappresenta la sorgente di clock principale del sistema. Fornisce anche la gestione del reset globale tramite reset synchronizer.
- **system_pll:** Phase-Locked Loop (PLL). Questo blocco permette di generare diverse frequenze di clock a partire dal clock in ingresso. Viene utilizzato per fornire i segnali di clock opportuni ai vari componenti del sistema (CPU, memoria, periferiche).
- **nios2_gen2:** soft-core CPU di Intel (Nios II). Esegue il software che controlla l'intero sistema e interagisce con le periferiche. Al fine di soddisfare i requisiti di performance imposti dalla gestione dello schermo grafico LT24, si è optato per la versione "fast" del computer.
- **onchip_memory:** memoria integrata all'interno dell'FPGA.

- **sdram controller:** controller per la memoria SDRAM esterna. Gestisce le operazioni di lettura e scrittura verso il chip di memoria SDRAM. In particolare ha la funzione di frame buffer per lo schermo LT24. Il processore scrive i dati dell'immagine nella SDRAM, e il controller dello schermo legge continuamente questi dati per pilotare il display.
- **jtag_uart:** UART che utilizza l'interfaccia JTAG per la comunicazione seriale con un computer host.
- **timer:** tale periferica viene utilizzata per gestire un'attesa pari a 1 s.
- **sysid_qsys:** fornisce un ID di sistema univoco e un numero di versione per il sistema embedded. Utile per l'identificazione hardware e software.
- **mm_clock_crosser:** utilizzato per interfacciare moduli Avalon che operano a frequenze di clock diverse. Fornisce un meccanismo sicuro per trasferire dati tra domini di clock asincroni.
- **pio_led, pio_sw e pio_key:** interfaccia di Input/Output Parallelo. Permette al processore di controllare i led presenti sulla scheda, leggere lo stato degli interruttori e gestire i pulsanti.
- **SEG7:** interfaccia per controllare i display a sette segmenti.
- **LT24_Controller:** controller per il display LT24 Terasic. Gestisce la generazione dei segnali necessari per pilotare il display LCD (dati, segnali di controllo, sincronizzazione).
- **LCD_RESET_n:** pin di output parallelo utilizzato per controllare il segnale di reset (attivo basso, indicato da _n) del display LCD.
- **i2c_0:** controller master I2C. Permette al sistema di comunicare con dispositivi slave I2C, come il sensore BME688.
- **spi_0:** interfaccia SPI configurata in modalità a 3 fili. Utilizzata per comunicare con la scheda SD.

4.2 Utilizzo delle Risorse

Di seguito viene riportato il **compilation report** ottenuto al termine della compilazione con Quartus.

Flow Status	Successful - Mon May 19 21:36:21 2025
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	3PROGETTO_SE
Top-level Entity Name	DE10_Lite_LT24_Painter
Family	MAX 10
Device	10M50DAF484C7G
Timing Models	Final
Total logic elements	6,005 / 49,760 (12 %)
Total registers	3812
Total pins	177 / 360 (49 %)
Total virtual pins	0
Total memory bits	165,296 / 1,677,312 (10 %)
Embedded Multiplier 9-bit elements	6 / 288 (2 %)
Total PLLs	1 / 4 (25 %)
UFM blocks	0 / 1 (0 %)
ADC blocks	0 / 2 (0 %)

Possiamo osservare che:

- Sono stati utilizzati 6005 logic elements pari al 12% di quelli disponibili.
- è stato utilizzato uno dei 4 PLL.
- Sono stati utilizzati 16 pin che corrispondono al 49% dei 360 disponibili necessari al funzionamento del sistema.
- Per quanto riguarda la memoria è stato utilizzato il 10% di spazio disponibile

4.3 Timing Analysis

L'analisi del timing è stata eseguita durante la fase di compilazione su Quartus tramite il Timing Analyzer. In questa fase, vengono analizzati tutti i segnali di clock presenti nel sistema, con l'obiettivo di verificare che ogni possibile percorso registro-logica-registro rispetti i vincoli di **setup** e **hold time**.

Clock	Setup	Hold	Recovery	Removal	Minimum Pulse Width
Worst-case Slack	1.030	0.125	3.352	0.467	4.620
ADC_CLK_10	N/A	N/A	N/A	N/A	96.000
MAX10_CLK1_50	16.722	0.150	18.100	0.467	9.330
MAX10_CLK2_50	N/A	N/A	N/A	N/A	16.000
altera_reserved_tck	44.551	0.147	47.996	0.568	49.412
u0 system_pll sd1 pll7 clk[0]	1.030	0.125	3.352	1.870	4.620
u0 system_pll sd1 pll7 clk[3]	42.730	0.141	95.030	1.715	49.678

5. Struttura del Software

Per quanto riguarda lo sviluppo del software, abbiamo scaricato i file che ci hanno fornito funzioni ad alto livello per la gestione dei vari dispositivi.

Dopo ogni misurazione, il sistema attende 3 secondi prima di proseguire, durante i quali non svolge alcuna altra operazione. Non utilizziamo quindi un *system tick*, poiché non è necessario eseguire attività parallele durante questo intervallo di tempo.

Di seguito riportiamo le funzioni utilizzate e la modalità con cui abbiamo gestito l'intero sistema.

5.1 Misure sensore BME688

La prima cosa da fare è definire la struttura che rappresenta il sensore e le sue funzioni. Il tipo è definito nella libreria Bosch bme68x.

```
struct bme68x_dev dev_bme;
```

Dopodichè si assegnano vari campi:

```
dev_bme.intf = BME68X_I2C_INTF;  
dev_bme.intf_ptr = i2c_0_handle;  
dev_bme.read = bme68x_i2c_read_wrapper;  
dev_bme.write = bme68x_i2c_write_wrapper;  
dev_bme.delay_us = bme68x_delay_us_wrapper;
```

In particolare:

- **intf**: tipo di interfaccia (nel nostro caso I2C)
- **intf_ptr**: puntatore all'handle I²C.
- **read**: funzione wrapper per leggere registri via I²C.
- **write**: funzione wrapper per scrivere in registri via I²C.
- **delay_us**: funzione wrapper per le attese.

Le funzioni wrapper servono ad adattare la libreria Bosch con l'hardware specifico del nostro sistema. In particolare abbiamo tre wrapper: uno di lettura, uno di scrittura e uno di attesa. Di seguito la spiegazione del funzionamento di ciascuna funzione:

- La funzione **bme68x_i2c_read_wrapper** effettua una lettura I2C da un registro del sensore. Viene chiamata internamente dalla libreria Bosch ogni volta che è necessario leggere dei dati (es. temperatura, pressione, ecc.). Usa la funzione HAL `alt_avalon_i2c_master_tx_rx()` per scrivere l'indirizzo del registro al sensore (TX) e leggere la risposta (RX).
- La funzione **bme68x_i2c_write_wrapper** effettua una scrittura I2C su uno o più registri del sensore. Viene utilizzata dalla libreria per configurare il sensore (oversampling, modalità operative, ecc.). Chiama `alt_avalon_i2c_master_transmit()` per inviare tutto in una transazione unica.
- La funzione **bme68x_delay_us_wrapper** introduce un ritardo in microsecondi tra una configurazione e una lettura (o tra scritture), come richiesto dalla sequenza operativa del sensore.

A questo punto si deve associare l'indirizzo I2C del nostro sensore tramite la seguente funzione:

```
alt_avalon_i2c_master_target_set(i2c_0_handle, BME68X_I2C_ADDR_LOW);
```

Questa funzione **imposta l'indirizzo del dispositivo di destinazione** sul bus I²C.

Si procede a questo punto con l'inizializzazione del sensore tramite una funzione definita in `utilities.c`:

```
initialize_sensor(&dev_bme);
```

Questa funzione a sua volta chiama la funzione di inizializzazione fornita dalla libreria del sensore Bosch: `bme68x_init(dev)`.

Fatta l'inizializzazione possiamo procedere con la misura dei valori:

```
rslt_bme = read_sensor_data(&dev_bme, &temp_out, &hum_out, &pres_out);
```

Anche questa funzione è definita in `utilities.c`. Si occupa della lettura dei dati ambientali dal sensore. Essa configura il sensore per una singola misurazione in **modalità "forced"** (effettua una sola misura e poi torna in standby), imposta i parametri di oversampling (tecnica che migliora la precisione delle misurazioni riducendo l'influenza del rumore,

attraverso la ripetizione delle letture della stessa grandezza e il calcolo della loro media) e il profilo del riscaldatore, esegue la misura e infine estrae i dati di temperatura, umidità, pressione e resistenza del gas.

5.2 Logging su SD

Per implementare il logging dei dati su una scheda SD, abbiamo utilizzato delle librerie preesistenti trovate in rete, progettate per interfacciarsi direttamente con la scheda SD tramite il protocollo SPI. In particolare, sono disponibili dei **wrapper** che includono le librerie `altera_avalon_spi_regs.h` e `altera_avalon_spi.h`, specifiche per i sistemi basati su Nios II. Queste librerie forniscono funzioni a basso livello che permettono di accedere ai registri del controller SPI. Di seguito riportiamo le principali istruzioni utilizzate, accompagnate da una breve descrizione:

- **IOWR_ALTERA_AVALON_SPI_SLAVE_SEL(SPI_0_BASE, 1);** Seleziona lo slave SPI su cui operare. In questo caso, il valore 1 seleziona il dispositivo connesso come primo slave sulla linea SPI.
- **IOWR_ALTERA_AVALON_SPI_CONTROL(SPI_0_BASE, 0x400);** Configura il registro di controllo del modulo SPI. Il valore 0x400 abilita la modalità master e avvia la trasmissione.
- **IOWR_ALTERA_AVALON_SPI_TXDATA(SPI_0_BASE, outgoing);** Scrive un byte nel registro di trasmissione SPI. Il valore outgoing rappresenta il dato da inviare alla scheda SD.
- **IORD_ALTERA_AVALON_SPI_STATUS(SPI_0_BASE);** Legge il registro di stato del modulo SPI. Serve, ad esempio, per verificare se la trasmissione è completa o se ci sono errori.
- **IORD_ALTERA_AVALON_SPI_RXDATA(SPI_0_BASE);** Legge un byte ricevuto dal registro di ricezione SPI. Utilizzato per acquisire i dati inviati dalla scheda SD in risposta.

Proseguiamo adesso con la descrizione del programma per quanto riguarda il pilotaggio della scheda SD. Si parte con l'inizializzazione del file system.

```
initialize_efsl(&efsl);
```

Deve essere chiamata una sola volta all'avvio del sistema. In particolare, riceve un puntatore a una struttura `EmbeddedFileSystem`, che conterrà tutte le informazioni di stato e configurazione del filesystem. In particolare chiama a sua volta la funzione `efs_init(efsl, SPI_0_NAME)`. Quest'ultima inizializza il filesystem embedded tramite EFSL, utilizzando una specifica interfaccia SPI per comunicare con il dispositivo di memoria (scheda SD).

A questo punto se l'inizializzazione è andata a buon fine si chiama la funzione `stampa_inizio` che scrive sulla SD un **marcatore** (stringa) che segnala l'inizio di una nuova sessione di logging dei dati. La funzione apre il file chiamato "sensor_log.txt" in modalità append e scrive una stringa costante chiamata `SESSION_START_MARKER`. Infine chiama `fs_flushFs(...)` per salvare i dati su SD, e infine chiude il file.

A questo punto nel ciclo infinito viene chiamata la funzione **stampa_SD**. Questa registra su SD una riga di log contenente i valori letti dal sensore (temperatura, umidità, pressione), in formato leggibile. Questa funzione **apre** il file `sensor_log.txt` in modalità **append** ('a'), cioè se

il file non esiste, viene creato mentre se esiste, i nuovi dati vengono scritti in fondo. A questo punto **scrive** nel file la stringa preparata. Chiama `fs_flushFs(...)` per forzare il salvataggio dei dati nella memoria della scheda SD (sincronizzazione). Controlla che i byte scritti siano quelli previsti e che il flush sia andato a buon fine. Infine chiude il file con `file_fclose(...)`.

Per operare sul file ogni funzione definisce la seguente variabile:

```
File localLogFile;
```

In particolare **File** è un tipo di dato definito dalla **libreria EFSL**. Non è il FILE standard del C (`<stdio.h>`), ma una struttura specifica della EFSL usata per gestire file su dispositivi embedded, come una scheda SD. Questo serve per operare su un file (es. "sensor_log.txt") con le funzioni `file_fopen(...)`, `file_write(...)` e `file_fclose(...)`.

Abbiamo fatto in modo che il logging su SD sia attivo o disattivo a seconda della posizione dello slider 9. Inoltre, per evitare errori imprevisti, l'SD può essere rimossa solo quando la modalità di logging viene disattivata, ponendo lo slider 9 in posizione 0. Una volta rimessa l'SD lo slider può essere riportato in posizione 1 e a quel punto viene rimontato tutto il file system e viene ripresa l'attività di logging. Per fare ciò, nel codice si ha:

```
can_log_to_sd = sliders & MSK_SLIDER_9;

if(can_log_to_sd && old_can_log_to_sd)
    stampa_SD(&efsl, temp_out, hum_out, pres_out, log_buffer, sizeof(log_buffer));
if(!can_log_to_sd && old_can_log_to_sd)
    old_can_log_to_sd = can_log_to_sd;
if(can_log_to_sd && !old_can_log_to_sd){
    old_can_log_to_sd = can_log_to_sd;
    initialize_efsl(&efsl);
    stampa_inizio(&efsl);
    stampa_SD(&efsl, temp_out, hum_out, pres_out, log_buffer, sizeof(log_buffer));
}
```

Di seguito la spiegazione di questo pezzo di codice:

- **can_log_to_sd = sliders & MSK_SLIDER_9:** si usa un mascheramento bit a bit per leggere lo stato dello slider 9. Se quel bit è 1, allora `can_log_to_sd` diventa vero.
- **if (can_log_to_sd && old_can_log_to_sd):** questa è la condizione per cui il logging è continuamente attivo. In questo caso si chiama la funzione `stampa_SD()`.
- **if (!can_log_to_sd && old_can_log_to_sd):** è stato disattivato il logging. In questo caso si aggiorna semplicemente `old_can_log_to_sd = can_log_to_sd`.
- **if (can_log_to_sd && !old_can_log_to_sd):** Il logging è stato riattivato. Viene aggiornata la variabile `old_can_log_to_sd`. Viene reinizializzato il file system e si riprende ad effettuare il logging.

5.3 Controllo display LT24

Le librerie utili per la gestione del display sono:

```
#include "ILI9341.h"
#include "alt_video_display.h"
#include "simple_graphics.h"
```

Queste librerie forniscono le funzioni e le strutture dati per controllare il display.

- **ILI9341.h**: driver specifico per il controller del display.
- **alt_video_display.h**: supporto per la struttura di visualizzazione Altera.
- **simple_graphics.h**: funzioni di disegno semplificate (linee, rettangoli, testi, ecc.).

Dopodichè si procede con la definizione della struttura display:

```
alt_video_display Display;
```

Questa struttura contiene le informazioni di configurazione e stato del display. È usata dalle librerie `simple_graphics.h` e `alt_video_display.h`.

A questo punto si procede con la definizione delle impostazioni iniziali del display:

```
Display.interlace = 0;
Display.bytes_per_pixel = 2;
Display.color_depth = 16;
Display.height = SCREEN_HEIGHT;
Display.width = SCREEN_WIDTH;
```

Questa sezione configura i parametri video:

- **interlace = 0**: indica la modalità di visualizzazione.
- **bytes_per_pixel = 2**: significa 16 bit per pixel (2 byte, quindi 65536 colori).
- **color_depth = 16**: profondità di colore.
- **height e width**: altezza e larghezza dello schermo, costanti definite altrove.

A questo punto possiamo chiamare la funzione `LCD_Init()` che inizializza fisicamente il display (reset, setup, ecc.).

Se cambia la posizione dei **slider**, lo schermo viene cancellato (riempito di nero). Questo serve a "ripulire" la visualizzazione ogni volta che cambia la modalità di output sul display. Di seguito la tecnica implementata per fare ciò:

```
if(sliders != old_sliders){
    LCD_Init();
    LCD_Clear(BLACK_24);
    old_sliders = sliders;
}
```

Per la visualizzazione effettiva ogni volta viene chiamata una funzione "mostra_valori" (e sue varianti) la quale permette di mostrare sul display i valori passati come argomenti. Di seguito la struttura della funzione:

```
mostra_valori(temp_out, hum_out, pres_out);
```

Viene dichiarato un buffer per memorizzare le stringhe da stampare sul display. Ogni riga formatta una stringa da stampare usando i valori dei sensori nel seguente modo:

```
sprintf(buffer, "- Temp: %.2f C", temp);
```

La funzione utilizzata per stampare su display è:

```
vid_print_string_alpha(horiz_offset, vert_offset + 90, GREEN_24, BLACK_24, tahomabold_20, &Display, buffer);
```

5.5 Controllo modalità con switch

In base allo **stato degli slider**, il programma mostra sul display i valori ambientali:

```
if(sliders & MSK_SLIDER_0){           //Modalità temperatura
    sliders_real = (sliders & (MSK_SLIDER_0 | MSK_SLIDER_9));
    WRITE_LEDR(sliders_real);
    mostra_valori_temp(temp_max, temp_mid, temp_min);
}
else if(sliders & MSK_SLIDER_1){      //Modalità umidità
    sliders_real = (sliders & (MSK_SLIDER_1 | MSK_SLIDER_9));
    WRITE_LEDR(sliders_real);
    mostra_valori_hum(hum_max, hum_mid, hum_min);
}
else if(sliders & MSK_SLIDER_2){      //Modalità pressione
    sliders_real = (sliders & (MSK_SLIDER_2 | MSK_SLIDER_9));
    WRITE_LEDR(sliders_real);
    mostra_valori_pres(pres_max, pres_mid, pres_min);
}
else{                                  //Modalità default (valori attuali)
    WRITE_LEDR(sliders);
    mostra_valori(temp_out, hum_out, pres_out);
}
```

In particolare, per capire quale è la configurazione e quindi cosa mostrare sullo schermo abbiamo usato delle maschere. Si capisce che lo slider 0, per come è strutturato questo pezzo di codice, è quello a maggiore priorità.

5.6 Memoria

Dalla seguente immagine si vede l'occupazione in memoria delle sezioni .text, .data e .bss del nostro eseguibile, con un totale di 305920 byte in decimale (0x4ab00 in esadecimale).

text	data	bss	dec	hex
291452	14092	376	305920	4ab00

Come possiamo vedere la sezione che occupa maggiore spazio in memoria è la text. Tutte le sezioni sono mappate nella SDRAM:

Linker Section Name	Linker Region Name	Memory Device Name
.bss	sdram	sdram
.entry	reset	onchip_memory
.exceptions	sdram	sdram
.heap	sdram	sdram
.rodata	sdram	sdram
.rwdata	sdram	sdram
.stack	sdram	sdram
.text	sdram	sdram

6. Verifica Raggiungimento degli Obiettivi

Sono stati raggiunti sia gli obiettivi primari che quelli secondari del progetto. Per quanto riguarda la fase di **logging su SD**, abbiamo verificato che il file viene creato e riempito correttamente. Di seguito è riportata una rappresentazione del contenuto del file generato:

```
----- Inizio Misurazioni -----  
  
Temp: 22.79 C, Hum: 72.85 %, Pres: 1.00 hPa  
Temp: 22.76 C, Hum: 72.84 %, Pres: 1.00 hPa  
Temp: 22.78 C, Hum: 72.83 %, Pres: 1.00 hPa  
Temp: 22.79 C, Hum: 72.79 %, Pres: 1.00 hPa  
Temp: 22.79 C, Hum: 72.78 %, Pres: 1.00 hPa  
Temp: 22.81 C, Hum: 72.78 %, Pres: 1.00 hPa  
Temp: 22.82 C, Hum: 72.82 %, Pres: 1.00 hPa  
Temp: 22.84 C, Hum: 72.89 %, Pres: 1.00 hPa  
Temp: 22.85 C, Hum: 72.95 %, Pres: 1.00 hPa  
Temp: 22.86 C, Hum: 73.07 %, Pres: 1.00 hPa
```

Per quanto riguarda la **visualizzazione su display**, il risultato ottenuto è soddisfacente: grazie agli switch è possibile modificare dinamicamente i dati mostrati. Di seguito viene illustrato cosa appare sul display in caso di configurazione con tutti gli switch di selezione abbassati:



Naturalmente, il progetto può essere ulteriormente migliorato. Ad esempio, si potrebbe sostituire l'uso della funzione `usleep` con il **System Tick Timer**, così da poter eseguire altre operazioni durante le fasi di attesa.

Non è stato possibile valutare la quantità di VOC. Il sensore consente la misurazione della resistenza del gas, la quale è ausiliaria per la valutazione della qualità dell'aria. Per tale valutazione, occorrono delle funzioni complesse messe a disposizione nelle librerie fornite da Bosch. Tuttavia i file sorgente contenenti le definizioni di tali funzioni, non sono compatibili con il processore NIOS 2.